



S.

COMPREHENSIVE PYTHON FUNDAMENTALS STUDY MATERIAL FOR M.TECH STUDENTS

Executive Summary

This study material provides an in-depth, academically rigorous guide to seven fundamental Python topics essential for M.Tech-level computer science education. Developed to serve as both a learning resource and reference document, the material covers Python's historical development, program structure best practices, language mechanics (identifiers, keywords, operators), type system, and input/output operations.

1. HISTORY AND APPLICATION AREAS OF PYTHON

Historical Development

Python's trajectory from its 1989 conception by Guido van Rossum at CWI (Netherlands) to its current status as the world's most popular programming language in 2026 reflects deliberate evolution toward simplicity and versatility.^{[1] [2] [3] [4]}

Critical Milestones:

1.0 Release (January 1994): Established foundational features including lambda functions, functional programming constructs (map, filter, reduce), and exception handling—mechanisms that remain central to Python's design philosophy over three decades later.^[1]

2.0 Era (2000-2020): Transformed Python from academic curiosity to enterprise-grade language. List comprehensions introduced elegant syntax for collection transformation; garbage collection and Unicode support enabled enterprise deployment. This period saw adoption across scientific computing (NumPy, SciPy) and web frameworks (Django).^{[5] [6] [1]}

3.x Transition (2008-Present): The deliberate break from 2.x represented commitment to modernization. Python 2's end-of-life in January 2020 marked completion of largest coordinated migration in programming history. Python 3 introduced async/await (3.5+), type hints (3.5+), and f-strings (3.6+), bringing Python to feature parity with contemporary languages while maintaining its simplicity philosophy.^{[2] [3] [4]}

Current State (2026): Python maintains top position on TIOBE Index, surpassing both Java and C. This dominance reflects genuine technical superiority in specific domains rather than marketing; the TIOBE ranking aggregates multiple objective metrics (search engine results, job postings, GitHub activity).^[2]

Application Domains

Python's versatility stems from language design emphasizing readability over performance—a trade-off that rarely matters for computational bottlenecks while dramatically reducing development time across domains.^{[1] [2]}

Domain	Applications	Technical Rationale
Data Science	Analysis, visualization, statistical modeling	NumPy/Pandas provide C-level performance with Python ergonomics
Machine Learning	TensorFlow, PyTorch, Scikit-learn implementations	Excellent matrix operations; GPU integration seamless
Web Development	Django, FastAPI, Flask backend services	Rapid prototyping; excellent async support in modern versions
Scientific Computing	Research code, simulations, mathematical modeling	Rich numerical libraries; publication-quality visualization
Automation	DevOps, system administration, task scheduling	String manipulation excellence; cross-platform scripting
Game Development	Indie games, simulation engines, physics engines	Pygame mature ecosystem; integration with C++ engines
Cybersecurity	Penetration testing, network analysis, tool development	Excellent for quick prototyping; strong in TCP/IP handling

2. STRUCTURE OF PYTHON PROGRAM

Canonical Program Structure (PEP 8 Compliance)

Professional Python programs follow rigorous organizational conventions codified in PEP 8 (Python Enhancement Proposal 8), the official style guide adopted across industry and academia.^[7]

Structural Organization (Top-to-Bottom):

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Module-level docstring using triple-quoted strings.
Describes module purpose, major features, and usage examples.
Lines typically limited to 72 characters for readability.
"""

# Import organization (with blank lines between groups)
import sys
import os
from pathlib import Path

import numpy as np
import pandas as pd

from mypackage.mymodule import my_function

# Module-level constants (UPPER_SNAKE_CASE)
DEFAULT_TIMEOUT = 30
MAX_RETRIES = 3

# Class definitions
class DataProcessor:
    """Class docstring."""
    class_var = "shared state"

    def __init__(self):
        self.instance_var = []

    def process(self, data):
        """Method implementation."""
        return self._internal_method(data)

    def _internal_method(self, data):
        """Internal helper method (leading underscore)."""
        return data

# Function definitions
def process_file(filepath):
    """
    Process a file and return results.

    Args:
        filepath: Path to input file

    Returns:
```

```

        Processed data as list

    Raises:
        FileNotFoundError: If file doesn't exist
    """
    with open(filepath) as f:
        return [line.strip() for line in f]

# Entry point guard (crucial for importability)
def main():
    """Main program execution."""
    data = process_file("input.txt")
    processor = DataProcessor()
    results = processor.process(data)
    print(f"Processed {len(results)} items")

if __name__ == "__main__":
    main()

```

PEP 8 Structural Requirements

Indentation: 4 spaces (never tabs; mixing causes `SyntaxError`). Maximum line length 79 characters ensures readability across diverse editor widths and enables side-by-side code review. [\[7\]](#)

Blank Line Usage:

- Two blank lines surround top-level functions and classes
- One blank line between methods within classes
- Multiple blank lines (sparingly) separate logical sections
- Blank lines within functions indicate logical groupings

Import Organization (Mandatory):

1. Standard library imports (`os`, `sys`, `pathlib`)
 2. Third-party imports (`numpy`, `pandas`, `requests`)
 3. Local application imports (`mymodule`, `mypackage`)
- Separate groups with blank lines; sort alphabetically within groups

3. IDENTIFIERS AND KEYWORDS

Identifier Rules and Constraints

An identifier in Python must conform to specific syntactic rules that prevent ambiguity in parsing and execution. [\[8\]](#) [\[9\]](#) [\[10\]](#)

Hard Constraints:

1. **Initial Character:** Must be letter (a-z, A-Z) or underscore (`_`)

- ✓ `student_name`, `_private`, `DatabaseConnection`
- ✗ `1name`, `9to5`, `@attribute`

2. Subsequent Characters: Alphanumerics (0-9) and underscore only

- ✓ `student_name123`, `var_2024`
- ✗ `student-name`, `var@2024`, `stu dent`

3. Case Sensitivity: Python distinguishes case

- `Name`, `name`, `NAME` are three distinct identifiers
- Critical in large codebases to maintain consistency

4. Keyword Prohibition: Cannot use Python's 35 reserved keywords (Python 3.11)

- ✗ `for = 10`, `if = True`, `class = MyClass`
- Attempting to use keywords raises `SyntaxError` at parse time

5. No Whitespace: Identifiers cannot contain spaces or special characters

- ✓ `my_variable`, `myVariable`
- ✗ `my variable`, `my-variable`

Keyword Classification and Usage

Control Flow Keywords (8 total): `if`, `elif`, `else`, `for`, `while`, `break`, `continue`, `pass`

```
if condition:
    for item in items:
        if invalid(item):
            continue
        process(item)
```

Function/Class Definition (5 total): `def`, `class`, `lambda`, `yield`, `return`

```
def calculate():
    return value

class DataProcessor:
    pass

transform = lambda x: x ** 2
```

Logical Operators (6 total): `and`, `or`, `not`, `in`, `is`, `True`, `False`, `None`

```
if x > 0 and y < 10:
    if value is None:
        result = False
```

Exception Handling (5 total): `try`, `except`, `finally`, `raise`, `assert`

```
try:
    risky_operation()
except Exception as e:
    handle_error(e)
finally:
    cleanup()
```

Module/Scope (6 total): `import`, `from`, `as`, `global`, `nonlocal`, `del`

Utility (3 total): `with`, `async`, `await`

Checking Keywords Programmatically:

```
import keyword

# Query single keyword
keyword.iskeyword("while")      # True
keyword.iskeyword("variable")   # False

# List all keywords
print(keyword.kwlist)           # Complete list
```

Naming Convention Standards (PEP 8)

Identifier Type	Convention	Example	Rationale
Modules	lowercase_with_underscores	<code>data_processor.py</code>	Imports become readable: <code>from data_processor import func</code>
Packages	lowercase_no_underscores	<code>mypackage/</code>	Package names in imports should be concise
Classes	CapWords (CamelCase)	<code>StudentRecord</code> , <code>DataProcessor</code>	Instantiation clearly marks class: <code>obj = MyClass()</code>
Functions	snake_case	<code>calculate_total()</code> , <code>process_data()</code>	Calls read naturally: <code>process_data(items)</code>
Variables	snake_case	<code>student_count</code> , <code>max_iterations</code>	Consistency with functions
Constants	UPPER_SNAKE_CASE	<code>MAX_STUDENTS</code> , <code>DEFAULT_TIMEOUT</code>	Visual distinction signals immutability
Private	<code>_leading_underscore</code>	<code>_internal_method()</code>	Convention signals "internal use"
Magic/Dunder	double	<code>init</code>, <code>str</code>	Reserved for Python special methods

4. OPERATORS AND PRECEDENCE

Complete Operator Taxonomy

Arithmetic Operators (Standard mathematical operations)

```
a, b = 17, 5

a + b      # 22 (addition)
a - b      # 12 (subtraction)
a * b      # 85 (multiplication)
a / b      # 3.4 (true division, returns float)
a // b     # 3 (floor division, returns int)
a % b      # 2 (modulus, remainder)
a ** b     # 1419857 (exponentiation)
-a         # -17 (unary negation)
+a         # 17 (unary positive)
```

Key Point: Division (/) always returns float; floor division (//) returns integer. This distinction matters for type-sensitive operations.

Comparison Operators (Evaluate to boolean)

```
5 == 5     # True (equality)
5 != 3     # True (inequality)
5 < 10     # True (less than)
5 <= 5     # True (less than or equal)
10 > 5     # True (greater than)
10 >= 10   # True (greater than or equal)

# Chained comparisons (Python-specific efficiency)
1 < 2 < 3  # Equivalent to (1 < 2) and (2 < 3) = True
5 > 3 > 1  # Equivalent to (5 > 3) and (3 > 1) = True
```

Logical Operators (Short-circuit evaluation crucial)

```
x = True
y = False

x and y    # False (both must be True)
x or y     # True (at least one must be True)
not x      # False (negation)

# Short-circuit evaluation (stops when result determined)
True or risky_function()  # Never calls risky_function()
False and risky_function() # Never calls risky_function()
```

Performance Implication: Short-circuit evaluation means expensive operations can be protected:

```
if x != 0 and 100 / x > 50: # Safe: second condition not evaluated if x == 0
    process()
```

Membership Operators (Test collection membership)

```
items = [1, 2, 3, 4, 5]
3 in items          # True
6 in items          # False
6 not in items      # True

# String membership
"Python" in "I love Python"    # True
"Java" in "I love Python"     # False
```

Identity Operators (Test object identity, not equality)

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

a is b          # False (different objects, same content)
a is c          # True (same object reference)
a == b          # True (same content)
a is not b      # True

# Canonical use: None checking
if value is None: # Correct (identity check)
    pass

if value == None: # Works but non-idiomatic
    pass
```

Bitwise Operators (Operate at binary level)

```
a = 12    # Binary: 1100
b = 10    # Binary: 1010

a & b     # 8 (AND: 1000)
a | b     # 14 (OR: 1110)
a ^ b     # 6 (XOR: 0110)
~a        # -13 (NOT: inverts all bits)
a << 1    # 24 (left shift, multiply by 2)
a >> 1    # 6 (right shift, divide by 2)
```

Use Cases: Flags, permissions, hardware manipulation

```
# File permissions (Unix-style)
READ = 0b100
WRITE = 0b010
```

```
EXECUTE = 0b001
```

```
user_perms = READ | WRITE
is_readable = (user_perms & READ) != 0 # True
```

Operator Precedence Hierarchy

The operator precedence table represents evaluation order when multiple operators appear in a single expression. [\[11\]](#) [\[12\]](#) [\[13\]](#) [\[14\]](#)

Complete Precedence (Highest to Lowest):

Rank	Operator	Description	Associativity
1	()	Parentheses	Left-to-Right
2	**	Exponentiation	Right-to-Left*
3	+x, -x, ~x	Unary operators	Right-to-Left
4	*, /, //, %	Multiplication group	Left-to-Right
5	+, -	Addition/subtraction	Left-to-Right
6	<<, >>	Bitwise shifts	Left-to-Right
7	&	Bitwise AND	Left-to-Right
8	^	Bitwise XOR	Left-to-Right
9	\	Bitwise OR	Left-to-Right
10	Comparisons	==, !=, <, >, <=, >=	Left-to-Right
11	in, not in	Membership	Left-to-Right
12	is, is not	Identity	Left-to-Right
13	not	Boolean NOT	Right-to-Left
14	and	Boolean AND	Left-to-Right
15	or	Boolean OR	Left-to-Right
16	if-else	Ternary conditional	Right-to-Left
17	lambda	Lambda expression	Right-to-Left

*Right-to-left associativity for exponentiation means $2 ** 3 ** 2 = 2 ** (3 ** 2) = 512$, not $(2 ** 3) ** 2 = 64$

Precedence Examples:

```
# Example 1: Arithmetic precedence
result = 2 + 3 * 4      # 3 * 4 evaluated first → 14
result = (2 + 3) * 4    # Parentheses override → 20

# Example 2: Logical operator precedence
```

```

x = 5
result = x > 3 and x < 10 or x == 5
# Evaluates: (5 > 3 and 5 < 10) or (5 == 5)
#           = (True and True) or True
#           = True or True = True

# Example 3: Comparison chaining
result = 1 < 2 < 3      # (1 < 2) and (2 < 3) = True

# Example 4: Exponentiation associativity
result = 2 ** 3 ** 2     # 2 ** (3 ** 2) = 2 ** 9 = 512

```

Best Practice: Always use parentheses for clarity, even when unnecessary. Code readability surpasses saving keystrokes:

```

# Correct but dense
result = a > b and c < d or e == f

# Better: explicitly parenthesized
result = (a > b and c < d) or (e == f)

```

5. BASIC DATA TYPES AND TYPE CONVERSION

Type System Overview

Python uses dynamic typing—variables can reference objects of any type without explicit declaration. However, each object has an immutable type determined at creation. [\[15\]](#) [\[16\]](#) [\[17\]](#)

Numeric Types

Integer (int): Arbitrary-precision whole numbers

```

x = 42
y = -1000
z = 0
large = 1_000_000_000      # Underscores for readability

print(type(x))             # class 'int'>
print(isinstance(x, int))  # True

# Arithmetic
print(10 // 3)             # 3 (floor division)
print(10 % 3)              # 1 (modulus)
print(2 ** 10)             # 1024 (exponentiation)

```

Float (float): IEEE 754 double-precision floating-point

```

pi = 3.14159
temp = -40.5

```

```

scientific = 1.5e-3      # 0.0015 (scientific notation)
infinity = float('inf')
not_a_number = float('nan')

print(type(pi))          # lass 'float'>

# Floating-point precision issue (important!)
print(0.1 + 0.2)          # 0.30000000000000004 (not 0.3)
print(0.1 + 0.2 == 0.3)   # False

# Solution: use decimal module for financial calculations
from decimal import Decimal
print(Decimal('0.1') + Decimal('0.2')) # Decimal('0.3')

```

Complex (complex): Real and imaginary components

```

z = 3 + 4j
print(z.real)            # 3.0
print(z.imag)            # 4.0
print(abs(z))            # 5.0 (magnitude)

# Complex arithmetic
z1 = 3 + 4j
z2 = 1 + 2j
print(z1 + z2)           # (4 + 6j)
print(z1 * z2)           # (-5 + 10j)

```

String Type (str)

Immutable sequences of Unicode characters. Supports rich set of operations:[\[18\]](#)

```

s = "Python"

# Concatenation
s1 = "Hello"
s2 = "World"
s3 = s1 + " " + s2      # "Hello World"

# Repetition
"ab" * 3                # "ababab"

# Indexing (0-based)
s[0]                    # "P"
s[-1]                   # "n" (last character)

# Slicing
s[1:4]                  # "yth" (s[1], s[2], s[3])
s[::2]                  # "Pto" (every second character)
s[::-1]                 # "nohtyP" (reversed)

# Membership
"th" in "Python"        # True
"Java" in "Python"      # False

```

```
# Length and methods
len(s)           # 6
s.upper()        # "PYTHON"
s.lower()        # "python"
s.replace("P", "J") # "Jython"

# String formatting (modern f-strings, Python 3.6+)
name = "Alice"
age = 25
f"Name: {name}, Age: {age}" # "Name: Alice, Age: 25"
```

Boolean Type (bool)

Subclass of int; True equals 1, False equals 0:^[19]

```
flag = True
print(type(flag))      # class 'bool'>
print(int(True))       # 1
print(int(False))      # 0

# Truthiness evaluation (important!)
print(bool(0))         # False
print(bool(1))         # True
print(bool(-1))        # True
print(bool(""))        # False (empty string)
print(bool("text"))    # True (non-empty string)
print(bool([]))        # False (empty list)
print(bool([1, 2]))    # True (non-empty list)
print(bool(None))      # False (special None value)
```

Truthiness Rules: In boolean context, values evaluate as:

- **False:** 0, 0.0, "", [], (), {}, None
- **True:** All other values (including 1, -1, "False")

Collection Types

List (list): Mutable, ordered, heterogeneous

```
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, True, None]

# Mutable operations
numbers[0] = 10
numbers.append(6)      # [10, 2, 3, 4, 5, 6]
numbers.extend([7, 8])
numbers.pop()          # Removes and returns last element
numbers.remove(2)      # Removes first occurrence of 2
numbers.sort()         # Sorts in place
```

Tuple (tuple): Immutable, ordered

```
coords = (10, 20)
single = (5,)          # Note: comma required for single-element tuple

# Unpacking
x, y = coords          # x = 10, y = 20

# Extended unpacking (Python 3)
data = (1, 2, 3, 4, 5)
first, *middle, last = data # first=1, middle=[2,3,4], last=5
```

Dictionary (dict): Key-value pairs, mutable, ordered (Python 3.7+)

```
student = {"name": "John", "age": 20, "gpa": 3.8}

# Access
student["name"]          # "John"
student.get("city", "NYC") # "NYC" (default if not found)

# Mutation
student["age"] = 21      # Update existing
student["city"] = "Boston" # Add new key-value

# Iteration
for key, value in student.items():
    print(f"{key}: {value}")
```

Set (set): Unordered, unique elements, mutable

```
unique_numbers = {1, 2, 3, 3, 2} # Duplicates removed
print(unique_numbers)           # {1, 2, 3} (order unspecified)

# Set operations
s1 = {1, 2, 3}
s2 = {2, 3, 4}
s1 | s2          # {1, 2, 3, 4} (union)
s1 & s2          # {2, 3} (intersection)
s1 - s2          # {1} (difference)
s1 ^ s2          # {1, 4} (symmetric difference)
```

Type Conversion (Casting)

Implicit Type Coercion

Python automatically promotes types in compatible operations:

```
# int + float = float
result = 5 + 2.5          # 7.5 (float)
```

```
# bool treated as int
result = True + 5          # 6 (True == 1)
result = False + 10       # 10 (False == 0)

# Hierarchy: bool < int < float < complex
# Lower-rank type promoted to higher-rank type
```

Explicit Type Conversion

```
# int() - Truncates floats, converts strings
int("42")                 # 42
int(3.7)                   # 3 (truncates, doesn't round)
int(True)                  # 1
int("FF", 16)             # 255 (hex string conversion)

# float() - Converts numeric types to float
float("3.14")              # 3.14
float(10)                  # 10.0
float("-inf")              # -infinity
float("inf")               # +infinity

# str() - String representation
str(42)                    # "42"
str(3.14)                  # "3.14"
str([1, 2, 3])             # "[1, 2, 3]"

# bool() - Boolean evaluation
bool(0)                    # False
bool(1)                    # True
bool("")                   # False (empty string)
bool("hello")              # True (non-empty string)

# Collection conversions
list("hello")              # ['h', 'e', 'l', 'l', 'o']
tuple([1, 2, 3])           # (1, 2, 3)
set([1, 2, 2, 3])          # {1, 2, 3}
dict([('a', 1), ('b', 2)])  # {'a': 1, 'b': 2}
```

Type Checking Best Practices

```
# Correct: isinstance() (recommended)
x = 42
isinstance(x, int)         # True
isinstance(x, (int, float)) # True (multiple types)

# Avoid: type() comparison (fragile with inheritance)
type(x) == int             # Works but problematic with subclasses
type(x) is int             # Even worse

# Dynamic type checking (EAFP - Easier to Ask Forgiveness than Permission)
try:
    result = 10 / x
```

```
except ZeroDivisionError:
    result = float('inf')
```

6. STATEMENTS AND EXPRESSIONS

Conceptual Distinction

Statements: Execute instructions; may modify program state; don't produce usable values.

Expressions: Evaluate to values; can be composed with operators; produce results.

```
# Statement (no return value)
x = 10                                # Assignment statement

# Expression (produces value)
10 + 5                                # Arithmetic expression → 15
x > 5                                  # Comparison expression → True (if x=10)
```

Simple Statements

Expression Statements

```
2 + 3                                # Evaluates to 5 (result typically discarded)
len([1, 2, 3])                       # Evaluates to 3
print("hello")                       # Calls function (prints output)
```

Assignment Statements

```
# Single assignment
x = 10

# Multiple assignment (parallel)
x, y = 5, 10                        # x=5, y=10
a = b = c = 0                       # All assigned to 0

# Tuple unpacking
coordinates = (10, 20)
x, y = coordinates                  # x=10, y=20

# Extended unpacking
first, *middle, last = [1, 2, 3, 4, 5]
# first=1, middle=[2,3,4], last=5

# Augmented assignment
x += 5                              # x = x + 5
x -= 3                              # x = x - 3
x *= 2                              # x = x * 2
```

Pass Statement

```
# Placeholder for empty code blocks
if condition:
    pass                # Will implement later

class ImplementLater:
    pass

def stub_function():
    pass
```

Del Statement

```
x = 10
del x                # Delete variable

my_list = [1, 2, 3]
del my_list[^1]      # Delete element → [1, 3]
```

Compound Statements

Conditional Statements (if-elif-else)

```
# Basic if
if age >= 18:
    print("Adult")

# if-else
if score >= 50:
    status = "Pass"
else:
    status = "Fail"

# if-elif-else (multiple conditions)
if marks >= 80:
    grade = "A"
elif marks >= 70:
    grade = "B"
elif marks >= 60:
    grade = "C"
else:
    grade = "F"

# Ternary operator (conditional expression)
status = "Pass" if score >= 50 else "Fail"

# Nested conditionals
if x > 0:
    if x < 10:
        category = "small positive"
```

```

    else:
        category = "large positive"
else:
    category = "non-positive"

```

Boolean Operator Short-Circuit Evaluation:

```

# AND: stops at first False
x > 0 and 10 / x > 5          # 10/x never evaluated if x <= 0

# OR: stops at first True
x == 0 or 10 / x > 5          # 10/x never evaluated if x == 0

# Practical: Safe division
if divisor != 0 and result / divisor > threshold:
    process()

```

Loop Statements

While Loop (indefinite iteration):

```

# Basic while
count = 1
while count <= 5:
    print(count)
    count += 1
# Output: 1 2 3 4 5

# While with break
i = 0
while True:
    if i == 3:
        break
    print(i)
    i += 1
# Output: 0 1 2

# While with continue
i = 0
while i < 5:
    i += 1
    if i == 3:
        continue          # Skip printing 3
    print(i)
# Output: 1 2 4 5

# While-else (else if loop completes without break)
x = 0
while x < 3:
    print(x)
    x += 1
else:

```

```
print("Loop completed")
# Output: 0 1 2 Loop completed
```

For Loop (definite iteration):

```
# Iterate over sequence
for fruit in ["apple", "banana", "cherry"]:
    print(fruit)

# Range iteration (most common)
for i in range(5):          # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 8):       # 2, 3, 4, 5, 6, 7
    print(i)

for i in range(0, 10, 2):   # 0, 2, 4, 6, 8 (step by 2)
    print(i)

for i in range(10, 0, -1):  # Countdown
    print(i)

# Enumerate (index and value)
for idx, item in enumerate(["a", "b", "c"]):
    print(f"{idx}: {item}")
# Output: 0: a 1: b 2: c

# Nested loops
for i in range(3):
    for j in range(2):
        print(f"({i}, {j})")

# List comprehension (concise loop)
squares = [x**2 for x in range(5)] # [0, 1, 4, 9, 16]

# Comprehension with filter
evens = [x for x in range(10) if x % 2 == 0]
# [0, 2, 4, 6, 8]
```

Function Definition

```
# Basic function
def greet(name):
    """Function docstring describes purpose and parameters."""
    return f"Hello, {name}!"

# Default parameters
def welcome(name="Guest"):
    print(f"Welcome, {name}!")

welcome()          # "Welcome, Guest!"
welcome("Alice")   # "Welcome, Alice!"
```

```

# Multiple return values
def get_stats(numbers):
    return min(numbers), max(numbers), sum(numbers)

mini, maxi, total = get_stats([5, 2, 8, 1])

# *args (variable positional arguments)
def sum_all(*args):
    return sum(args)

sum_all(1, 2, 3, 4)          # 10

# **kwargs (keyword arguments)
def print_config(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_config(host="localhost", port=8000, debug=True)

```

Try-Except-Finally (Exception Handling)

```

# Basic try-except
try:
    x = int("not_a_number")
except ValueError:
    print("Invalid input")

# Multiple except blocks
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
except TypeError:
    print("Type error")

# Try-except-else (else if no exception)
try:
    file = open("data.txt")
    data = file.read()
except FileNotFoundError:
    print("File not found")
else:
    print(f"Read {len(data)} bytes")

# Try-except-finally (finally always executes)
try:
    operation()
except Exception as e:
    print(f"Error: {e}")
finally:
    cleanup()          # Executes regardless

```

7. INPUT/OUTPUT STATEMENTS

Output with print()

Basic Usage

```
# Single argument
print("Hello, World!")      # Hello, World!

# Multiple arguments (comma-separated)
print("Name:", "Alice")    # Name: Alice
print("x =", 10, "y =", 20) # x = 10 y = 20

# Print variables
x, y = 5, 10
print(x, y)                # 5 10
```

print() Parameters

```
# sep parameter (separator between arguments)
print("a", "b", "c", sep="-") # a-b-c
print("a", "b", "c", sep="")  # abc

# end parameter (string appended at end)
print("Loading", end="")
for i in range(3):
    print(".", end="", flush=True)
print(" Done!")
# Output: Loading... Done!

# Combining parameters
for i in range(3):
    print(i, sep=" ", end=" | ") # 0 | 1 | 2 |
```

String Formatting

f-Strings (Modern Standard, Python 3.6+):

```
name = "Alice"
age = 25
print(f"Name: {name}, Age: {age}") # Name: Alice, Age: 25

# Expressions in f-strings
x, y = 10, 20
print(f"Sum: {x + y}")             # Sum: 30

# Format specifications
pi = 3.14159
print(f"Pi: {pi:.2f}")             # Pi: 3.14 (2 decimal places)
print(f"Pi: {pi:.4f}")             # Pi: 3.1416
```

```
# Alignment and padding
name = "Bob"
print(f" | {name:>10} | ")      # |          Bob | (right-aligned)
print(f" | {name:<10} | ")      # | Bob           | (left-aligned)
print(f" | {name:^10} | ")      # |    Bob      | (centered)

# Number formatting
count = 1234567
print(f"Count: {count:,}")      # Count: 1,234,567
print(f"Percentage: {0.85:.1%}") # Percentage: 85.0%
```

.format() Method (Older, Still Valid):

```
print("Name: {}, Age: {}".format("Alice", 25))
print("Name: {0}, Age: {1}".format("Alice", 25))
print("Name: {name}, Age: {age}".format(name="Alice", age=25))
```

% Formatting (Legacy, Not Recommended):

```
print("Name: %s, Age: %d" % ("Alice", 25))
print("Value: %.2f" % 3.14159)      # Value: 3.14
```

Input with input()

Basic Usage

```
# input() reads from console, returns string
name = input("Enter name: ")
print(f"Hello, {name}!")

# input() always returns string (important!)
age_str = input("Enter age: ")
age = int(age_str)      # Must convert
```

Type Conversion of Input

```
# Convert to integer
age = int(input("Age: "))

# Convert to float
height = float(input("Height (m): "))

# Convert to boolean
response = input("Continue? (yes/no): ")
is_continuing = response.lower() == "yes"

# Multiple inputs on one line
x, y = map(int, input("Two numbers: ").split())
```

```
# Example: "5 10" → x=5, y=10

# Split and unpack
parts = input("Name and age: ").split()
name, age = parts[0], int(parts[1])
```

Input Validation

```
# Validate with while loop
while True:
    try:
        age = int(input("Age (0-120): "))
        if 0 <= age <= 120:
            break
        else:
            print("Age must be 0-120!")
    except ValueError:
        print("Enter a valid integer!")

print(f"Registered age: {age}")
```

Practical I/O Examples

Example 1: Calculator

```
print("=== Simple Calculator ===")
num1 = float(input("Number 1: "))
operator = input("Operator (+,-,*,/): ")
num2 = float(input("Number 2: "))

if operator == "+":
    result = num1 + num2
elif operator == "-":
    result = num1 - num2
elif operator == "*":
    result = num1 * num2
elif operator == "/":
    if num2 != 0:
        result = num1 / num2
    else:
        print("Error: Division by zero!")
        exit()
else:
    print("Invalid operator!")
    exit()

print(f"{num1} {operator} {num2} = {result:.2f}")
```

Example 2: Grade Calculation

```
print("=== Grade Calculation ===")
student_name = input("Name: ")
marks = int(input("Marks (0-100): "))

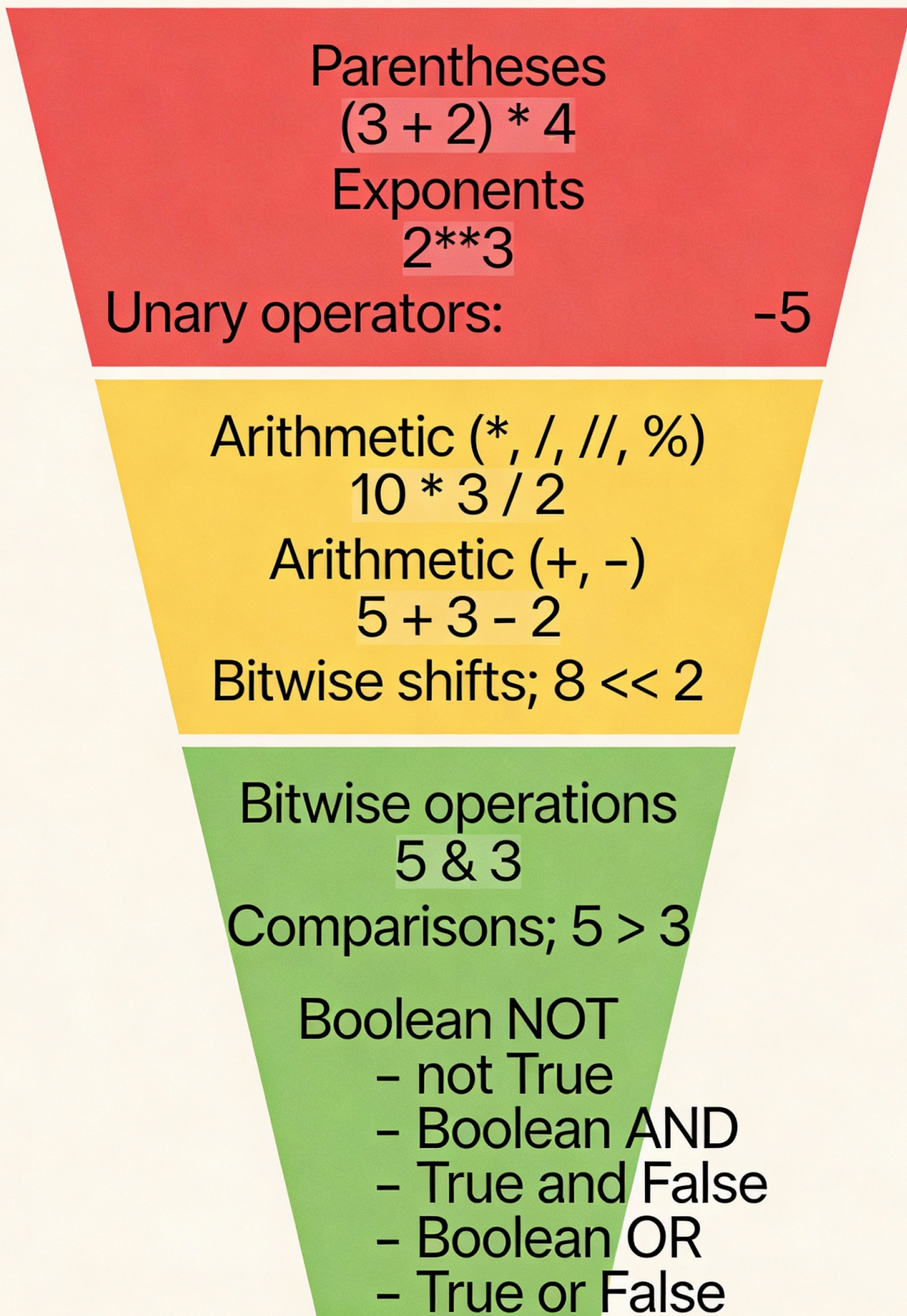
if marks >= 80:
    grade = "A"
elif marks >= 70:
    grade = "B"
elif marks >= 60:
    grade = "C"
elif marks >= 50:
    grade = "D"
else:
    grade = "F"

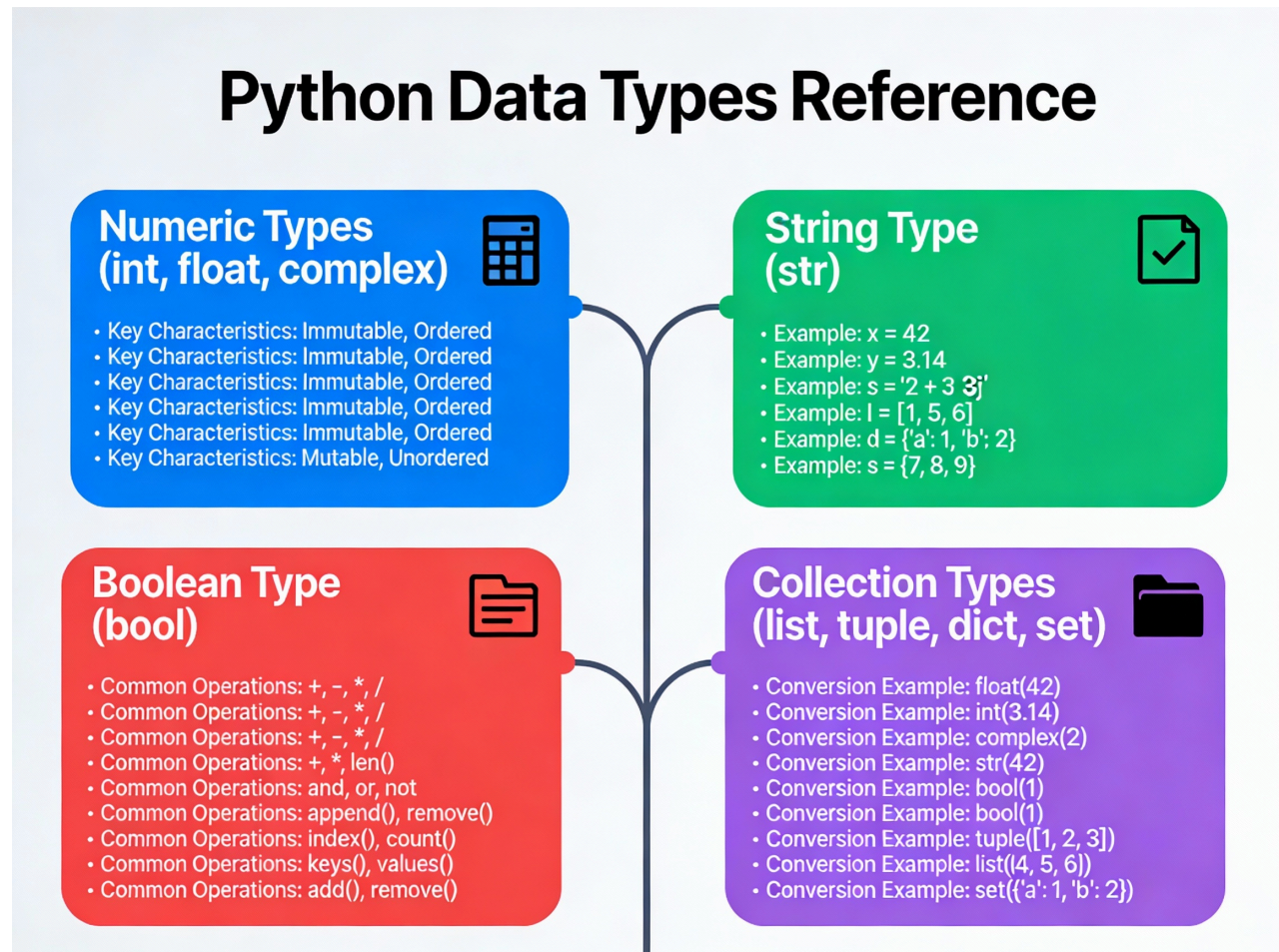
print(f"{student_name}: {marks}% → Grade {grade}")
```

VISUAL REFERENCES

![Python Operator Precedence Hierarchy]

M.Tech Pythonfundamental Study





Python Data Types Reference - Complete Guide with Examples

CONCLUSION AND RECOMMENDATIONS

This comprehensive study material covers seven fundamental Python topics essential for M.Tech-level computer science education. The material bridges theoretical understanding with practical application through:

Academic Rigor: Each concept includes formal definitions, mathematical notation, and theoretical foundations

Practical Applicability: Code examples demonstrate real-world usage patterns and best practices

Industry Standards: Content adheres to PEP 8 style guide and current Python 3.11+ conventions

Pedagogical Structure: Progressive complexity from language history through input/output operations

Study Recommendations

1. **Code Along:** Execute all examples in interactive Python REPL or Jupyter notebooks
2. **Modify Examples:** Change values, operators, and conditions to internalize behavior
3. **Reference:** Bookmark for quick lookup during projects and assignments
4. **Practice:** Complete exercises on operator precedence, type conversion, and control flow
5. **Integrate:** Apply PEP 8 standards and naming conventions in all assignments

Assessment Focus Areas

- Operator precedence and expression evaluation
- Type conversion and implicit coercion
- Conditional logic and loop control
- Exception handling mechanisms
- String formatting and input validation

REFERENCES

Bocasay (2024). History of Python Programming Language.^[1]
RealPython. Python Script Structure Best Practices.^[20]
Programiz. Python Keywords and Identifiers.^[8]
InfoWindTech (2025). History of Python: Origins, Growth, Modern Impact.^[2]
Reddit. Python Project Optimization.^[21]
GeeksforGeeks. Python Keywords and Identifiers.^[9]
GeeksforGeeks. Python Version History.^[3]
GeeksforGeeks. Structuring Python Programs.^[22]
UpGrad. Identifiers in Python: Naming Rules & Best Practices.^[10]
Wikipedia. Python Programming Language.^[23]
Dagster. Best Practices in Structuring Python Projects.^[24]
HeroVired. Identifiers in Python.^[25]
Wikipedia. History of Python.^[5]
TutorialsPoint. Python - History and Versions.^[4]
PEP 8. Style Guide for Python Code.^[7]
PythonGeeks. Operator Precedence in Python.^[11]
Programiz. Python Type Conversion.^[15]
DrBTaneja. Python Input and Output Functions.^[26]
Programiz. Precedence and Associativity of Operators.^[12]
DataCamp. Python Data Type Conversion Guide.^[16]
Programiz. Python Basic Input and Output.^[18]
W3Schools. Python Operator Precedence.^[13]
Shiksha. Type Conversion in Python.^[27]
GeeksforGeeks. Input and Output in Python.^[28]
StackOverflow. Logical Operator Precedence.^[14]
GeeksforGeeks. How To Convert Data Types in Python 3.^[29]

DCLessons. Python Input & Output Statements. ^[30]
Runestone Academy. Precedence of Operators. ^[31]
W3Schools. Python Data Types. ^[17]
W3Schools. Python Type Conversion. ^[32]
Tutorials Point. Python Operator Precedence. ^[33]
GeeksforGeeks. Type Conversion in Python. ^[34]
W3Schools. Python Input and Output. ^[35]
PythonForAll. Precedence & Evaluation Rules. ^[36]
LearnPython4CBSE. Python input() and print() Functions. ^[37]
YouTube. Conditional IF-ELSE and Loop Statements. ^[38]
Python Documentation. Simple Statements. ^[39]
RealPython. Python eval() Function. ^[40]
W3Schools. Python Conditions and If Statements. ^[19]
W3Schools. Python Statements. ^[41]
GeeksforGeeks. Evaluate Expression in String. ^[42]
GeeksforGeeks. Conditional Statements in Python. ^[43]
PYnative. Python Statements With Examples. ^[44]
StarterTutorials. Expression Evaluation in Python. ^[45]
Purdue University. Conditions and Loops Summary. ^[46]
TutorialsPoint. Python Basic Syntax. ^[47]
Python Official Documentation. Expressions. ^[48]
Wiingy. Python Syntax with Examples. ^[49]
GeeksforGeeks. Statement, Indentation, Comment in Python. ^[50]
W3Schools. Python While Loops. ^[51]
RealPython. Python while Loops. ^[52]

Study Material Created: January 2026

Version: 1.0 (Comprehensive)

Target Audience: M.Tech Computer Science Students

Scope: Python Fundamentals (Beginner to Intermediate)

Format: Structured Study Guide with Examples, Best Practices, and Visual

✱

1. <https://www.bocasay.com/history-python-programming/>
2. <https://www.infowindtech.com/the-history-of-python-origins-growth-its-modern-impact/>
3. <https://www.geeksforgeeks.org/python/python-version-history/>
4. https://en.wikipedia.org/wiki/History_of_Python
5. [https://en.wikipedia.org/wiki/Python_\(programming_language\)](https://en.wikipedia.org/wiki/Python_(programming_language))
6. https://www.tutorialspoint.com/python/python_history.htm
7. <https://peps.python.org/pep-0008/>
8. <https://www.programiz.com/python-programming/keywords-identifier>
9. <https://www.geeksforgeeks.org/python/python-keywords-and-identifiers/>
10. <https://www.upgrad.com/blog/identifiers-in-python-naming-rules-best-practices/>

11. <https://pythongeeks.org/python-operator-precedence/>
12. <https://www.programiz.com/python-programming/precedence-associativity>
13. https://www.w3schools.com/python/python_operators_precedence.asp
14. <https://stackoverflow.com/questions/16679272/priority-precedence-of-the-logical-operators-order-of-operations-for-not-an>
15. <https://www.programiz.com/python-programming/type-conversion-and-casting>
16. <https://www.datacamp.com/tutorial/python-data-type-conversion>
17. https://www.w3schools.com/python/python_datatypes.asp
18. <https://www.programiz.com/python-programming/input-output-import>
19. https://www.w3schools.com/python/python_conditions.asp
20. <https://realpython.com/python-script-structure/>
21. https://www.reddit.com/r/Python/comments/18qkivr/what_is_the_optimal_structure_for_a_python_project/
22. <https://www.geeksforgeeks.org/python/structuring-python-programs/>
23. <https://pythondev.readthedocs.io/history.html>
24. <https://docs.python-guide.org/writing/structure/>
25. <https://www.teachoo.com/19882/4182/Question-1/category/Short-Answer-Type-Questions-2-Marks-each/>
26. <https://drbtaneja.com/python-input-and-output-functions/>
27. <https://www.shiksha.com/online-courses/articles/type-conversion-in-python-types-and-examples/>
28. <https://www.geeksforgeeks.org/python/input-and-output-in-python/>
29. <https://www.geeksforgeeks.org/python/how-to-convert-data-types-in-python-3/>
30. <https://www.dcclasses.com/python-input-output-statements>
31. <https://runestone.academy/ns/books/published/fopp/Conditionals/PrecedenceofOperators.html>
32. [https://www.learnpython4cbse.com/python/4.-python:-input-\(\)-and-print-\(\)-function](https://www.learnpython4cbse.com/python/4.-python:-input-()-and-print-()-function)
33. https://www.tutorialspoint.com/python/python_operator_precedence.htm
34. <https://www.geeksforgeeks.org/python/type-conversion-in-python/>
35. <https://www.scaler.com/topics/python/python-input-and-output/>
36. <https://www.pythonforall.com/python/operators/precedence>
37. https://www.w3schools.com/python/gloss_python_type_conversion.asp
38. <https://www.youtube.com/watch?v=kgLo4X9j1Bc>
39. https://docs.python.org/3/reference/simple_stmts.html
40. <https://realpython.com/python-eval-function/>
41. https://www.w3schools.com/python/python_statements.asp
42. <https://www.geeksforgeeks.org/python/python-evaluate-expression-given-in-string/>
43. <https://www.geeksforgeeks.org/python/conditional-statements-in-python/>
44. <https://pynative.com/python-statements/>
45. <https://www.startertutorials.com/blog/expression-evaluation-python-programming.html>
46. https://www.purdue.edu/hla/sites/varalalab/wp-content/uploads/sites/20/2018/04/Lecture_11.pdf

47. https://www.tutorialspoint.com/python/python_basic_syntax.htm
48. <https://docs.python.org/3/reference/expressions.html>
49. <https://realpython.com/python-while-loop/>
50. <https://wiingy.com/learn/python/python-syntax/>
51. <https://www.geeksforgeeks.org/computer-science-fundamentals/expressions-in-python/>
52. https://www.w3schools.com/python/python_while_loops.asp
53. <https://dagster.io/blog/python-project-best-practices>
54. <https://herovired.com/home/learning-hub/topics/identifiers-in-python>
55. <https://discuss.python.org/t/python-project-structure/36119>
56. <https://blog.ksrdatavision.com/python/python-identifiers/>
57. <https://www.geeksforgeeks.org/python/python-syntax/>
58. https://courses.cs.washington.edu/courses/cse140/13wi/eval_rules.pdf
59. <https://openbookproject.net/books/bpp4awd/ch04.html>
60. <https://www.geeksforgeeks.org/python/statement-indentation-and-comment-in-python/>